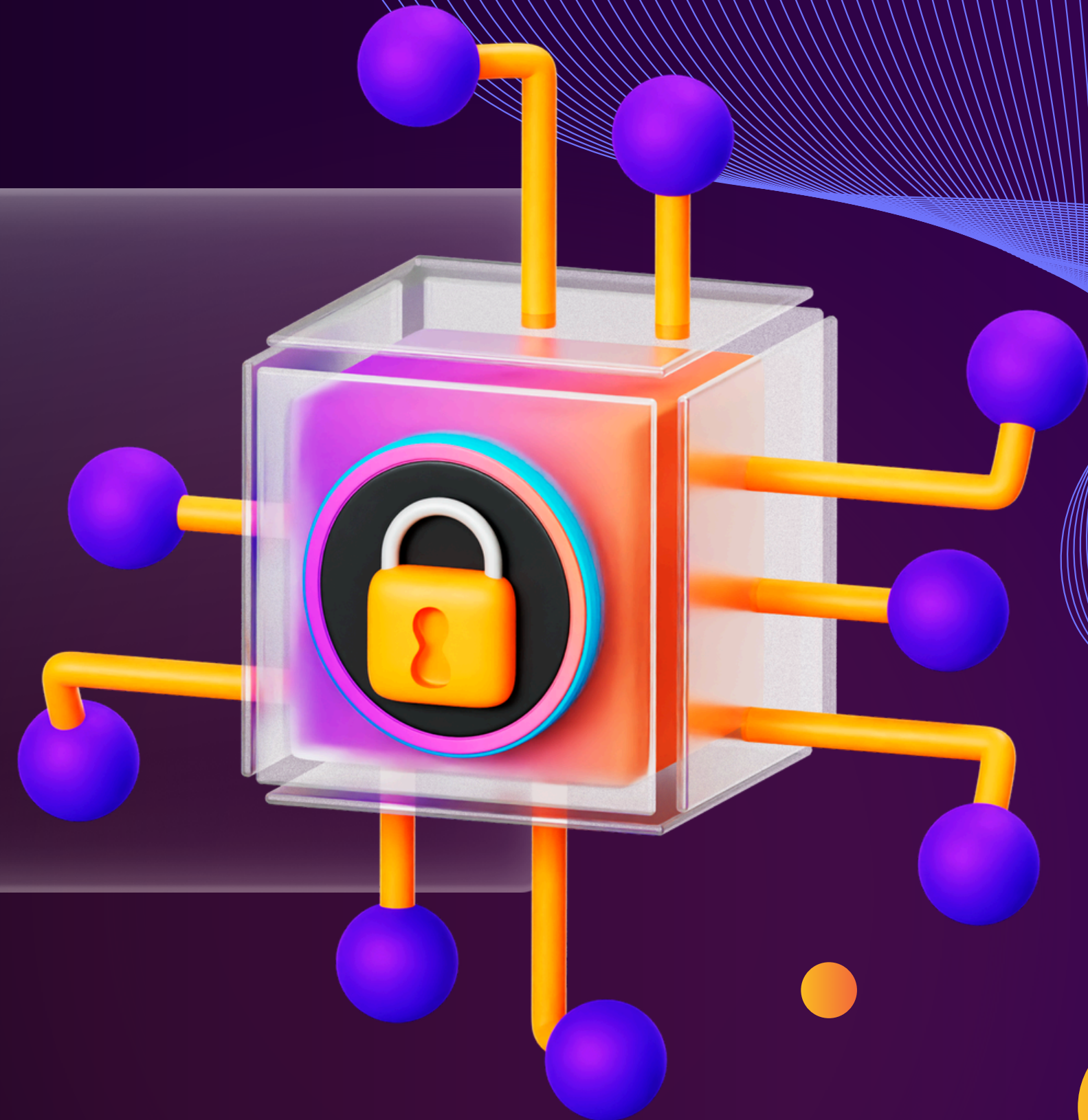




PREDICTX

FULL-STACK ON-CHAIN PREDICTION
MARKET PROTOCOL

Presented by : **Ingkar Adilbek, Kadirzhan Kozha &
Ansar Ibraev**



Blockchain Technologies 2



PROJECT SCOPE

PredictX is an on-chain binary prediction market protocol deployed on Base Sepolia.

Users trade YES/NO outcome shares through a constant-product AMM with *0.3% fees, slippage protection, LP tokens, and ERC-1155 outcome tokens*. Markets support liquidity provision, oracle-based resolution, and collateral claims for winning share holders.

The protocol includes a CREATE/CREATE2 market factory, UUPS V1 → V2 upgrade path, ERC-4626 FeeVault, Chainlink oracle adapter with stale-price validation, and DAO governance through ERC20Votes, Governor, and a 2-day Timelock.

The full-stack system also includes a React dApp, The Graph subgraph, verified Base Sepolia deployment, Slither audit checks, CI/CD, and a Foundry test suite with unit, fuzz, invariant, fork, gas, governance, and upgrade tests.

SYSTEM ARCHITECTURE

Architecture Flow:

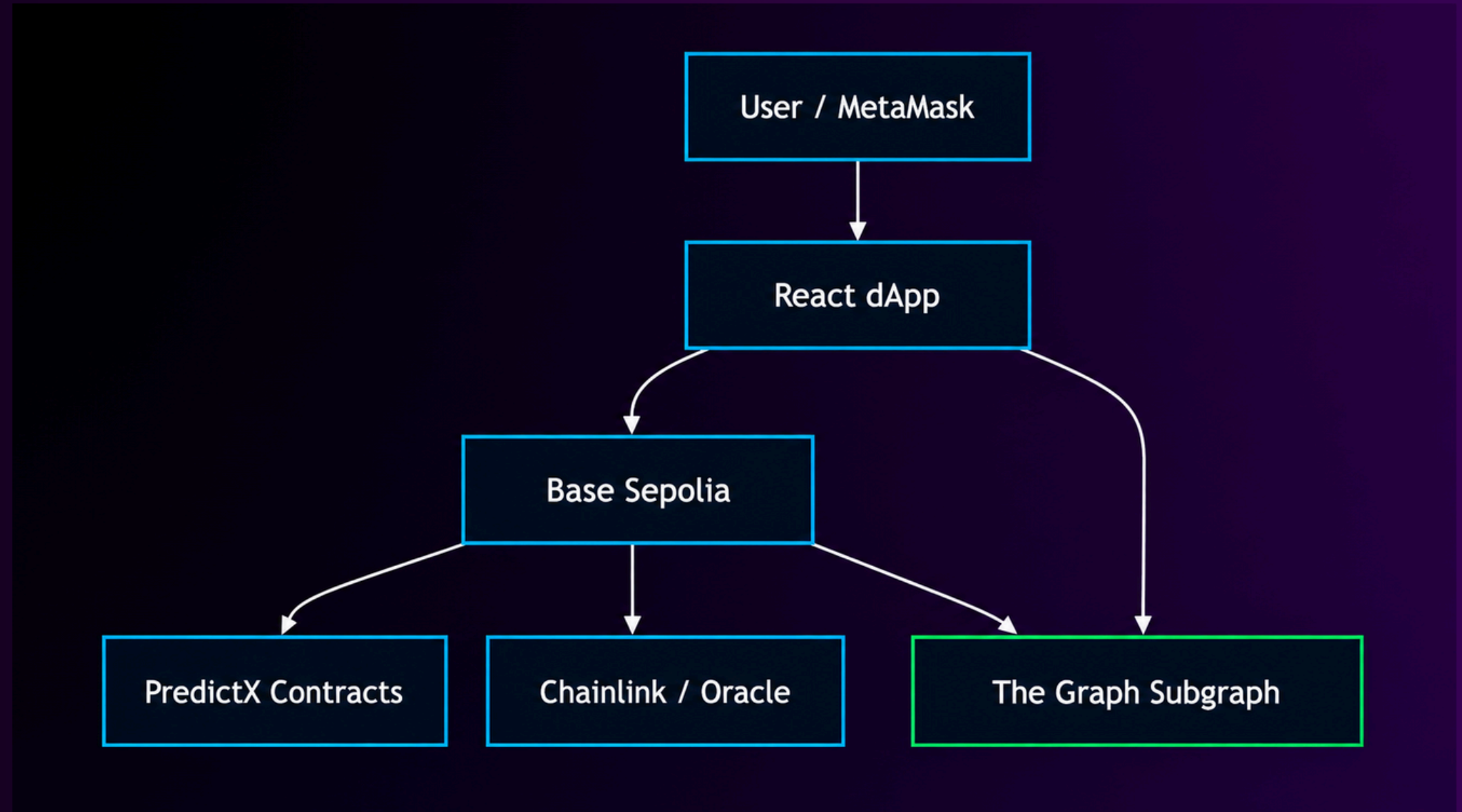
- React dApp connects MetaMask and targets Base Sepolia.
- Frontend reads balances, voting power, delegate address, vault shares, governance parameters, and subgraph-indexed proposal/history data.
- Write actions include collateral minting, vault deposit flow, vote delegation, and proposal state/vote interactions.

On-Chain Protocol:

- Factory deploys markets through CREATE and CREATE2.
- PredictionMarket runs CPMM trading with 0.3% fees, slippage protection, LP accounting, ERC1155 YES/NO shares, oracle resolution, and reward claims.
- GovernanceToken, Governor, and 2-day Timelock manage protocol decisions.

Data & Indexing:

- Chainlink adapter validates oracle freshness.
- The Graph indexes protocol events for market history, trades, liquidity events, claims, and governance proposals.



SMART CONTRACT SYSTEM

All components are deployed and verified on Base Sepolia.



MARKET LAYER

- PredictionMarket
- PredictionMarketFactory
- MarketTypes
- AMMMath



TOKEN LAYER

- OutcomeToken
ERC1155
- LPToken ERC20
- GovernanceToken
ERC20Votes + Permit



GOVERNANCE LAYER

- ProtocolGovernor
- ProtocolTimelock
- 2-day delay



INFRASTRUCTURE

- FeeVault ERC4626
- ChainlinkOracleAdapter
- MockOracleAdapter /
Tests
- YulMath

PATTERNS DEMONSTRATED

1. **Factory pattern** in PredictionMarketFactory: it allows markets to be deployed consistently and supports both normal CREATE deployment and deterministic CREATE2 deployment. This means future market addresses can be predicted before deployment, which is useful for frontend integration, indexing, and reproducible protocol behavior.
2. **UUPS upgradeability** through PredictionMarketUpgradeable: implemented and tested a documented V1 to V2 upgrade path, including role-restricted upgrade authorization. This proves that the protocol can support future logic upgrades while protecting the upgrade process through access control.
3. **Checks-Effects-Interactions** followed by the trading and liquidity functions use ReentrancyGuard where needed: this protects flows like buying shares, selling shares, adding liquidity, removing liquidity, and claiming rewards from common reentrancy risks.
4. **AccessControl roles** protect the privileged operations: Minting, resolving markets, upgrading implementations, and depositing protocol fees are all role-gated. This avoids unguarded admin functions and makes the permission model clear for audit and governance.
5. **Oracle adapter pattern, Timelock governance, and a state-machine market lifecycle**: oracle logic is separated from market logic, governance actions pass through a 2-day Timelock, and markets move through controlled states such as Open and Resolved. These patterns make the system more modular, auditable, and safer to extend.



AMM & OUTCOME SHARES

01 CPMM PRICING

Each market has two reserves: YES and NO. PredictX uses constant-product pricing:

$$\underline{x} * \underline{y} = k$$

Buying YES adds collateral to the YES side and removes shares from the opposite reserve. This changes the reserve ratio and updates the implied market price.

02 TRADING PROTECTIONS

buyYes / buyNo:

- transfer collateral from trader
- calculate sharesOut with AMMMath
- apply 0.3% fee
- check minSharesOut for slippage protection
- update reserves
- mint ERC1155 YES/NO outcome shares

sellYes / sellNo:

- burn outcome shares
- calculate collateralOut
- check minCollateralOut
- transfer collateral back to trader

03 TOKEN MODEL

OutcomeToken:

ERC1155 shares for each market and outcome.
Token ID = hash(marketId, outcome).

LPToken:

ERC20 token minted when users add liquidity and burned when liquidity is removed.

Winning shares are burned during claimRewards, and collateral is paid to the winning holder.

ORACLE & INDEXING

The screenshot shows the Subgraph Studio interface for a subgraph named "predictx-base-sepolia". The top navigation bar includes "Subgraph Studio", "My Dashboard", "API Keys", "Billing", "Docs", "Support", and a user profile "0x838f-0A609d". The subgraph details section shows a status of "DEPLOYED", indexing on "Base Sepolia", a slug of "predictx-base-sepolia", a deploy key "f4b6de-f54d22", and an entity count of "0". A progress bar indicates "100% SYNCED". The "Details" tab is active, showing a display name "predictx-base-sepolia" and a subgraph description "My Final". A "Getting Started" section provides instructions on how to deploy the subgraph to Subgraph Studio successfully, with buttons for "Endpoints" and "Documentation". A "Select a network" dropdown menu is visible, showing "Base Sepolia" as the selected network and "TESTNET" as the environment.

01 ORACLE INTEGRATION

ChainlinkOracleAdapter validates:

- answeredInRound $\geq$ roundId
- startedAt and updatedAt are non-zero
- updatedAt is not in the future
- price is greater than zero
- price is not stale beyond configured delay

MockOracleAdapter supports deterministic testnet/demo resolutions.

02 SUBGRAPH INTEGRATION

Deployed subgraph:

[https://api.studio.thegraph.com/query/1753352/predictx-base-sepolia/ v0.0.1](https://api.studio.thegraph.com/query/1753352/predictx-base-sepolia/v0.0.1)

Indexes events from:

- PredictionMarketFactory
- PredictionMarket templates
- ProtocolGovernor

Entities include markets, trades, liquidity events, users, reward claims, protocol stats and governance proposals.

DAO GOVERNANCE

Governor + Timelock + ERC20Votes

1 day

Voting Delay

1 week

Voting Period

4%

Quorum

1%

Proposal Threshold

2 days

Timelock delay

DELEGATE

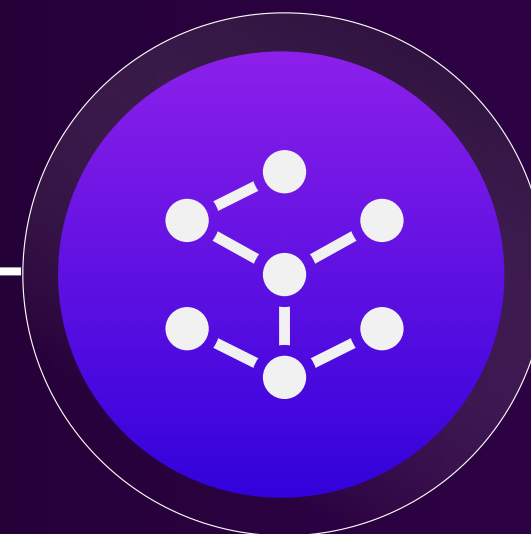
PROPOSE

VOTE

QUEUE

TIMELOCK

EXECUTE



SECURITY & AUDIT SUMMARY

0

SLITHER HIGH

0

SLITHER MEDIUM

0

TEST FAILURES

0

INVARIANT
FAILURES

Severity	Count
Critical	0
High	0
Medium	0
Low	0
Informational	2
Gas	0

```

ingkarpahr@MacBook-Air-Ingkar BT2_final % python3 -m slither . --config-file slither.config.json
/Users/ingkarpahr/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'.
See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
INFO:Slither:slither.config.json has an unknown key: checklist : False
'forge clean' running (wd: /Users/ingkarpahr/Desktop/BT2_final)
'forge config --json' running
'forge build --build-info --skip ./test/** ./script/** --force' running (wd: /Users/ingkarpahr/Desktop/BT2_final)
INFO:Detectors:
Detector: timestamp
PredictionMarket.resolveMarket() (src/core/PredictionMarket.sol#280-295) uses timestamp for comparisons
  Dangerous comparisons:
  - block.timestamp < resolutionTime (src/core/PredictionMarket.sol#281)
ChainlinkOracleAdapter.latestPrice() (src/oracle/ChainlinkOracleAdapter.sol#38-59) uses timestamp for comparisons
  Dangerous comparisons:
  - startedAt == 0 || updated == 0 || updated > block.timestamp (src/oracle/ChainlinkOracleAdapter.sol#46)
  - block.timestamp - updated > stalePriceDelay (src/oracle/ChainlinkOracleAdapter.sol#54)
MockOracleAdapter.getResolution(bytes32) (src/oracle/MockOracleAdapter.sol#57-65) uses timestamp for comparisons
  Dangerous comparisons:
  - block.timestamp > resolution.updatedAt + maxStaleness (src/oracle/MockOracleAdapter.sol#62)
MockOracleAdapter.isResolved(bytes32) (src/oracle/MockOracleAdapter.sol#67-75) uses timestamp for comparisons
  Dangerous comparisons:
  - block.timestamp <= resolution.updatedAt + maxStaleness (src/oracle/MockOracleAdapter.sol#74)
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#block-timestamp
INFO:Detectors:
Detector: unindexed-event-address
Event IGovernor.ProposalCreated(uint256,address,address[],uint256[],string[],bytes[],uint256,uint256,string) (lib/openzeppelin-contracts/contracts/governance/IGovernor.sol#118-128) has address parameters but no indexed parameters
Event GovernorTimelockControl.TimelockChange(address,address) (lib/openzeppelin-contracts/contracts/governance/extensions/GovernorTimelockControl.sol#31) has address parameters but no indexed parameters
Event IERC1967.AdminChanged(address,address) (lib/openzeppelin-contracts/contracts/interfaces/IERC1967.sol#18) has address parameters but no indexed parameters
Reference: https://github.com/crytic/slither/wiki/Detector-Documentation#unindexed-event-address-parameters
INFO:Slither:. analyzed (99 contracts with 97 detectors), 7 result(s) found

```

SECURITY CONTROLS

- ReentrancyGuard on buy/sell, liquidity, and claim flows
- SafeERC20 for all collateral transfers
- AccessControl roles: RESOLVER, MINTER, UPGRADER, DEFAULT_ADMIN
- CEI ordering before external transfers
- Slippage checks: minSharesOut / minCollateralOut
- Oracle validation: stale price, future timestamp, incomplete round
- Timelock delay: 2 days before governed execution

REVIEWED RISKS

- Reentrancy during trading, liquidity removal, and reward claims
- Unauthorized minting, resolving, or upgrading
- Oracle stale data and incomplete Chainlink rounds
- Reserve depletion and impossible AMM output
- Governance abuse through proposal spam or fast execution
- Timestamp dependence in deadlines and oracle freshness
- ERC20 transfer failures and unsafe return handling

AUDIT ARTIFACTS

- audits/security-audit.md: full internal audit report
- SECURITY.md: disclosure and security policy
- slither.config.json: static-analysis configuration
- coverage/coverage-report.md: contract coverage report
- gas-reports/gas-report.md: gas benchmark report
- GitHub Actions: forge test, coverage, Slither, frontend, subgraph

TESTING & COVERAGE

Ran 20 test suites in 41.75s (172.39s CPU time): 172 tests passed, 0 failed, 0 skipped (172 total tests)

File	% Lines	% Statements	% Branches	% Funcs
script/Deploy.s.sol	0.00% (0/28)	0.00% (0/40)	100.00% (0/0)	0.00% (0/1)
script/DeployMocks.s.sol	0.00% (0/9)	0.00% (0/12)	100.00% (0/0)	0.00% (0/1)
script/VerifyDeployment.s.sol	0.00% (0/30)	0.00% (0/43)	0.00% (0/20)	0.00% (0/1)
src/core/PredictionMarket.sol	91.54% (119/130)	91.85% (124/135)	54.17% (13/24)	100.00% (10/10)
src/core/PredictionMarketFactory.sol	100.00% (23/23)	95.65% (22/23)	66.67% (2/3)	100.00% (5/5)
src/core/PredictionMarketUpgradeable.sol	87.50% (14/16)	85.71% (12/14)	0.00% (0/2)	100.00% (4/4)
src/governance/ProtocolGovernor.sol	80.95% (17/21)	80.95% (17/21)	100.00% (0/0)	80.00% (8/10)
src/libraries/AMMMath.sol	100.00% (13/13)	100.00% (17/17)	100.00% (3/3)	100.00% (3/3)
src/libraries/YulMath.sol	87.50% (7/8)	80.00% (4/5)	0.00% (0/1)	100.00% (3/3)
src/mocks/MockChainlinkAggregator.sol	72.22% (13/18)	69.23% (9/13)	100.00% (0/0)	80.00% (4/5)
src/mocks/MockERC20.sol	100.00% (9/9)	66.67% (6/9)	0.00% (0/3)	100.00% (3/3)
src/mocks/MockGovernanceTarget.sol	100.00% (4/4)	100.00% (3/3)	100.00% (0/0)	100.00% (1/1)
src/mocks/PredictionMarketUpgradeableV2.sol	100.00% (4/4)	100.00% (2/2)	100.00% (0/0)	100.00% (2/2)
src/oracle/ChainlinkOracleAdapter.sol	80.95% (17/21)	83.33% (20/24)	33.33% (2/6)	100.00% (3/3)
src/oracle/MockOracleAdapter.sol	100.00% (25/25)	92.86% (26/28)	71.43% (5/7)	100.00% (6/6)
src/tokens/GovernanceToken.sol	100.00% (12/12)	93.33% (14/15)	75.00% (3/4)	100.00% (4/4)
src/tokens/LPToken.sol	100.00% (12/12)	92.86% (13/14)	80.00% (4/5)	100.00% (3/3)
src/tokens/OutcomeToken.sol	86.36% (19/22)	80.00% (24/30)	42.86% (3/7)	83.33% (5/6)
src/vault/FeeVault.sol	100.00% (11/11)	83.33% (10/12)	33.33% (1/3)	100.00% (3/3)
test/gas/YulMathGas.t.sol	100.00% (12/12)	100.00% (12/12)	100.00% (0/0)	100.00% (6/6)
test/invariant/PredictionMarketHandler.t.sol	87.18% (34/39)	84.09% (37/44)	70.00% (7/10)	100.00% (7/7)
test/unit/AMMMath.t.sol	100.00% (6/6)	100.00% (5/5)	100.00% (0/0)	100.00% (3/3)
Total	78.44% (371/473)	72.36% (377/521)	43.88% (43/98)	92.22% (83/90)

Testing Results

- 172 tests passed
- 0 failed
- 0 skipped

Coverage

- PredictionMarket.sol: 91.54%
- Contract-scope coverage: 91.40%

The 91.40% metric is calculated across src/ runtime contracts:

319 covered lines / 349 total lines = 91.40%

Suite includes unit, fuzz, invariant, fork, gas, governance, and upgrade tests.



UNIT TESTS

Ran 12 test suites in 7.22ms (15.73ms CPU time): 140 tests passed, 0 failed, 0 skipped (140 total tests)

- PredictionMarket buy/sell, liquidity, resolution, claims
- Factory CREATE and CREATE2 market deployment
- ERC1155 outcome shares and ERC20 LP tokens
- ERC4626 FeeVault deposit/withdraw flows
- Chainlink oracle validation and governance lifecycle
- UUPS V1 → V2 upgrade authorization



FUZZ & INVARIANT TESTS

Ran 3 test suites in 60.74ms (130.38ms CPU time): 12 tests passed, 0 failed, 0 skipped (12 total tests)

Invariant tests validate:

- reserves never become zero
- collateral covers tracked balances
- LP supply does not exceed collateral supply
- outcome token supply stays bounded
- market remains open during trading sequences

Ran 1 test suite in 17.38s (17.37s CPU time): 5 tests passed, 0 failed, 0 skipped (5 total tests)

Ran 1 test suite in 20.54ms (10.51ms CPU time): 6 tests passed, 0 failed, 0 skipped (6 total tests)



FORK TESTS

Ran 3 test suites in 1.63s (4.13s CPU time): 9 tests passed, 0 failed, 0 skipped (9 total tests)

Fork tests interact with real mainnet protocols to prove external integration behavior:

- USDC metadata, supply, whale balance and transfer
- Uniswap V2 router factory, WETH path and getAmountsOut
- Chainlink ETH/USD feed round data, decimals and description

GAS REPORT & L1 VS L2 COSTS

GAS MEASUREMENT

Measured with:

- forge snapshot
- forge build --sizes
- Base Sepolia deployment broadcast logs

Report: [gas-reports/gas-report.md](#)

OPTIMIZATION EVIDENCE

Benchmarked:

- Yul vs Solidity helpers
- CREATE vs CREATE2 deployment
- ERC1155 outcome-share model
- ERC4626 vault operations
- bytecode sizes under EIP-170 limit

Largest runtime:

- ProtocolGovernor — 15,535 bytes
- Limit: 24,576 bytes

L1 VS BASE SEPOLIA

Compared 6+ operations:

- deploy GovernanceToken
- deploy ProtocolGovernor
- deploy FeeVault
- buyYes
- sellYes
- addLiquidity
- createMarket

Base Sepolia keeps the same EVM execution model with much lower ETH-denominated transaction cost than Ethereum L1.

Operation	Gas Units	L1 Cost Profile	Base Sepolia
Deploy Governor	2.39M	High	Low
Deploy FeeVault	1.27M	High	Low
buyYes	~145k	Medium	Low
sellYes	~168k	Medium	Low
addLiquidity	~121k	Medium	Low
createMarket	~1.65M	High	Low

GITHUB ACTIONS + CI PIPELINE

Workflow Trigger

- Runs automatically on every push
- Runs automatically on every pull request
- Workflow file: `.github/workflows/ci.yml`

Job 1: Contracts and Slither

- Checks out repository with recursive submodules
- Installs Foundry (stable)
- Runs `forge --version`
- Installs contract dependencies with `forge install`
- Validates Solidity formatting using `forge fmt --check`
- Builds contracts with `forge build --sizes`
- Executes full test suite with `forge test -vvv`
- Generates gas snapshot with `forge snapshot`
- Generates coverage with `forge coverage`
- Installs Python 3.11
- Installs Slither
- Runs static analysis with `slither . --config-file slither.config.json`

Job 2: Subgraph

- Depends on successful contracts job
- Installs Foundry and builds contracts for ABI artifacts
- Installs Node.js 20
- Runs `npm ci` in subgraph/
- Generates types with `npm run codegen`
- Builds subgraph with `npm run build`

Job 3: Frontend

- Depends on successful contracts job
- Installs Node.js 20
- Checks formatting with `npm run format:check`
- Validates production build with `npm run build`

Final Result

- Contracts and Slither passed
- Subgraph passed
- Frontend passed
- Repository is validated across smart contracts, indexing, and UI

All checks have passed

3 successful checks

✓



CI / Contracts and Slither (push)

Successful in 5m

Details

✓



CI / Frontend (push)

Successful in 19s

Details

✓



CI / Subgraph (push)

Successful in 2m

Details

chore: ignore generated local artifacts

 KadKoml committed 2 days ago · ✓ 3 / 3

docs: add deployed subgraph endpoint

 KadKoml committed 2 days ago · ✓ 3 / 3

feat(frontend): add governance proposal list

 KadKoml committed 2 days ago · ✓ 3 / 3

ci: run slither in contracts job

 KadKoml committed 2 days ago · ✓ 3 / 3

Here we show the CI pipeline configured in GitHub Actions. The workflow runs on every push and pull request. It has three validation jobs. The first job checks Solidity formatting, builds contracts, runs the full Foundry test suite, generates gas and coverage reports, and performs Slither static analysis. The second job validates the subgraph by rebuilding contract artifacts, generating Graph types, and compiling the subgraph. The third job validates the frontend by installing dependencies, checking formatting, and producing a production build. In the final repository state, all three jobs passed successfully.

DEPLOYMENT & VERIFICATION

Base Sepolia deployment registry



DEPLOYMENT SCRIPTS

- script/DeployMocks.s.sol
- script/Deploy.s.sol
- script/VerifyDeployment.s.sol

Contracts verified on BaseScan after broadcast deployment.

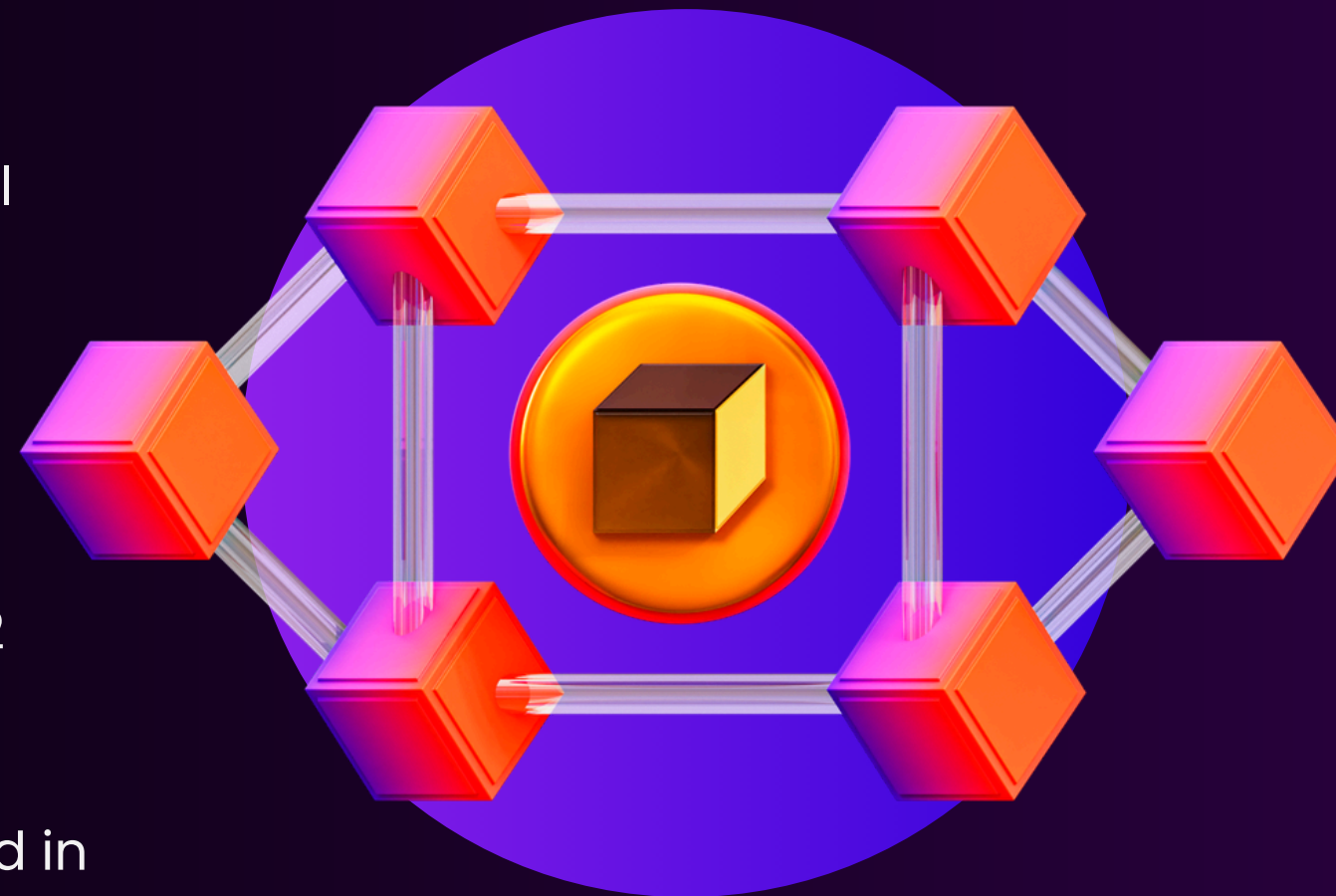


NETWORK

- Base Sepolia Chain ID: 84532
- Deployment registry:

deployments/addresses.md

Subgraph endpoint documented in README and subgraph README.



VERIFIED CONTRACTS

- GovernanceToken **0x2F6E...7d03**
- OutcomeToken **0xF1E2...C869**
- LPToken **0xf720...Bf3F**
- FeeVault **0x8E7e...E275**
- Timelock **0x5943...074C**
- Governor **0x77b8...dCF8**
- Factory **0xCF2A...00aE**
- MockERC20 **0xbA42...E8D2**
- MockOracle **0x95E9...9AAC**

FRONTEND DAPP

IMPLEMENTED UI FEATURES

- MetaMask wallet connection
- Base Sepolia wrong-network detection
- Readable transaction error messages
- Token balance reads Voting power reads
- Delegate address reads
- Vault shares and managed assets
- Factory deployment/status section
- Subgraph proposal and protocol event data
- Governance proposal list and vote button

WRITE TRANSACTIONS

1. Mint mock collateral
2. Approve + deposit into ERC4626 FeeVault
3. Delegate governance voting power
4. Cast governance vote by proposal ID/list

FRONTEND LAYOUT OVERVIEW

Displayed on-chain data:

- wallet address and chain ID
- collateral balance
- governance balance and voting power
- delegate address
- vault shares and managed assets
- governance parameters
- factory / market status

Supported write actions:

- mint mock collateral
- approve + deposit into the ERC4626 FeeVault
- delegate voting power
- load proposal state and cast governance votes by proposalId

The UI also detects wrong networks and shows readable transaction errors.

BASE SEPOLIA PREDICTION MARKET PROTOCOL

PredictX

On-chain binary prediction markets with CPMM trading, ERC1155 outcome shares, ERC4626 vault accounting, Governor + Timelock DAO, and The Graph indexing.

Wallet

Connect MetaMask

Protocol Overview

Markets Created0

Collateral Balance0

Governance Balance0

Voting Power0

Vault Shares0

Vault Managed Assets0

Vault & Collateral

Write transactions: mint mock collateral, approve vault, deposit into ERC4626 vault.

Mint amount

1000

Mint Mock Collateral

Deposit amount

10

Approve + Deposit

Governance

GovernorPredictX Governor

Voting Delay7200

Voting Period50400

Proposal Threshold10000.0

Delegatee address

0x...

Delegate Voting Power

Proposal ID

Proposal ID

Load Proposal State

Vote

For

Cast Vote

Markets

Factory deployment is live on Base Sepolia. Market creation support is prepared through the deployed factory contract.

Markets Count0

Full create-market UI can be wired to the factory once market parameters are finalized for demo flow.

The Graph Indexing

Subgraph data loaded.

Governance Proposals

No indexed governance proposals yet.

BASE SEPOLIA PREDICTION MARKET PROTOCOL

PredictX

On-chain binary prediction markets with CPMM trading, ERC1155 outcome shares, ERC4626 vault accounting, Governor + Timelock DAO, and The Graph indexing.

Wallet

Connected account

0x838f...609d

Chain ID

84532

Protocol Overview

Markets Created

0

Collateral Balance

2100.0

Governance Balance

1000000.0

Voting Power

1000000.0

Vault Shares

0.0

Vault Managed Assets

0.0

Delegate

0x838fDEB9F549D6515A1d517763b82E420c0A609d

Vault & Collateral

Write transactions: mint mock collateral, approve vault, deposit into ERC4626 vault.

Mint amount

1000

Mint Mock Collateral

Deposit amount

10

Approve + Deposit

MetaMask

Account 7

0x838fD...A609d

\$126.21

+\$0.00 (+0.00%)

Discover

Buy

Swap

Send

Receive

Tokens

Perps

DeFi

NFTs

Activity

Base Sepolia

May 17, 2026

Mint

Confirmed

-0 ETH

-\$0.00

May 16, 2026

Mint

Confirmed

-0 ETH

-\$0.00

Delegate

Confirmed

-0 ETH

-\$0.00

Mint

Confirmed

-0 ETH

-\$0.00

Governance

Governor

PredictX Governor

Voting Delay

7200

Voting Period

50400

Proposal Threshold

10000.0

Delegatee address

0x838fdeb9f549d6515a1d517763b82e420c0a609d

Delegate Voting Power

Protocol Overview

Markets Created

0

Collateral Balance

2090.0

Governance Balance

1000000.0

Voting Power

1000000.0

Vault Shares

10.0

Vault Managed Assets

10.0

Delegate

0x838fDEB9F549D6515A1d517763b82E420c0A609d

Governance

Governor

PredictX Governor

Voting Delay

7200

Voting Period

50400

Proposal Threshold

10000.0

Delegatee address

0x838fDEB9F549D6515A1d517763b82E420c0A609d

Delegate Voting Power

Proposal ID

483147680280551739475462145529029841448129

Load Proposal State

Proposal state: Pending

Vote

For

Cast Vote

Governance Proposals

Governance proposals loaded from subgraph.

Proposal ID

48314768028055173947546214552902984144812981027415071482929356650676633766421

Demo proposal: read FeeVault total managed assets for PredictX frontend governance testing

Proposer

0x838f...609d

Vote Window

41632758 → 41683158

Load On-Chain State

Selected Proposal

48314768028055173947546214552902984144812981027415071482929356650676633766421

On-chain state: Pending

Vote

For

Cast Vote

Governance

Governor

PredictX Governor

Voting Delay

7200

Voting Period

50400

Proposal Threshold

10000.0

Delegatee address

0x...

Delegate Voting Power

Proposal ID

483147680280551739475462145529029841448129

Load Proposal State

Proposal state: Active

Vote

For

Cast Vote

Governance

Governor

PredictX Governor

Voting Delay

7200

Voting Period

50400

Proposal Threshold

10000.0

Delegatee address

Ox...

Delegate Voting Power

Proposal ID

483147680280551739475462145529029841448129

Load Proposal State

Proposal state: Succeeded

Vote

For

Cast Vote

Governance Proposals

Governance proposals loaded from subgraph.

Proposal ID

48314768028055173947546214552902984144812981027415071482929356650676633766421

Demo proposal: read FeeVault total managed assets for PredictX frontend governance testing

Proposer

0x838f...609d

Vote Window

41632758 → 41683158

Load On-Chain State

Selected Proposal

48314768028055173947546214552902984144812981027415071482929356650676633766421

On-chain state: Active

Vote

For

Cast Vote

Governance Proposals

Governance proposals loaded from subgraph.

Proposal ID

48314768028055173947546214552902984144812981027415071482929356650676633766421

Demo proposal: read FeeVault total managed assets for PredictX frontend governance testing

Proposer

0x838f...609d

Vote Window

41632758 → 41683158

Load On-Chain State

Selected Proposal

48314768028055173947546214552902984144812981027415071482929356650676633766421

On-chain state: Succeeded

Vote

For

Cast Vote

The frontend verifies the full governance read flow:

- proposalId from the indexed ProposalCreated event
- proposer and description from the subgraph
- vote window from Governor data
- live proposal state from ProtocolGovernor
- voting options and Cast Vote action through MetaMask

Live demonstrated state:

- Pending proposal loaded from The Graph and ProtocolGovernor

Governance lifecycle covered by tests:

- Active voting
- Succeeded / Defeated result
- Queue and execute through Timelock

This proves that governance, subgraph indexing, voting power delegation, and frontend state loading are integrated end-to-end.

TEAM CONTRIBUTIONS

Ingkar

Responsible for:

- Contracts + scripts
- Core protocol contracts, deployment scripts, verification flow, Base Sepolia deployment support.

Branch:

- *ingkar-init*

Ansar

Responsible for:

- Testing + coverage
- Unit, fuzz, invariant, fork, gas and upgradeability test coverage.

Branch:

- *ansar-test*

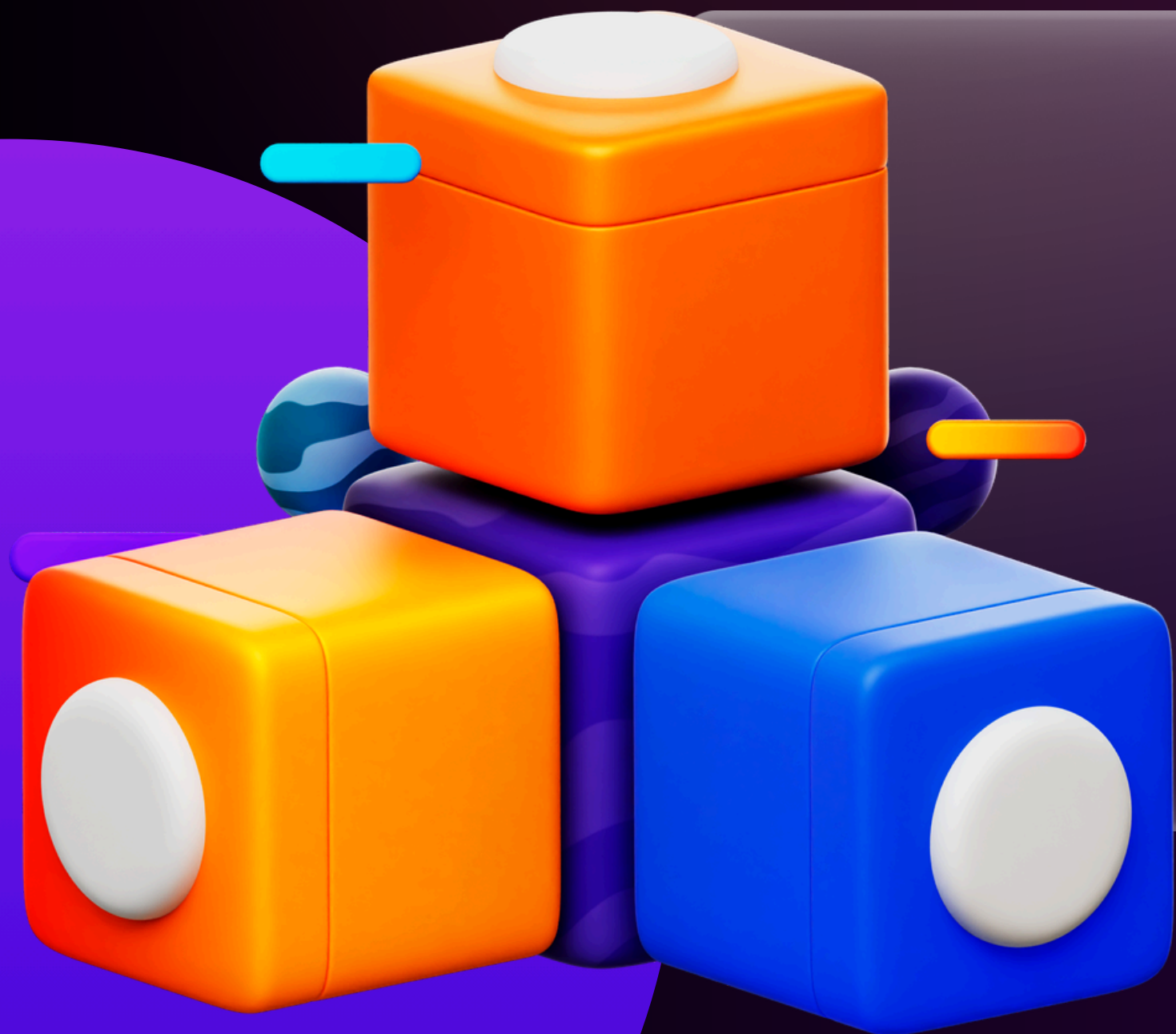
Kadirzhan

Responsible for:

- Frontend + documentation
- React frontend, subgraph/frontend integration, README and project documentation polish.

Branch:

- *kadirzhan-ui*



THANK YOU

